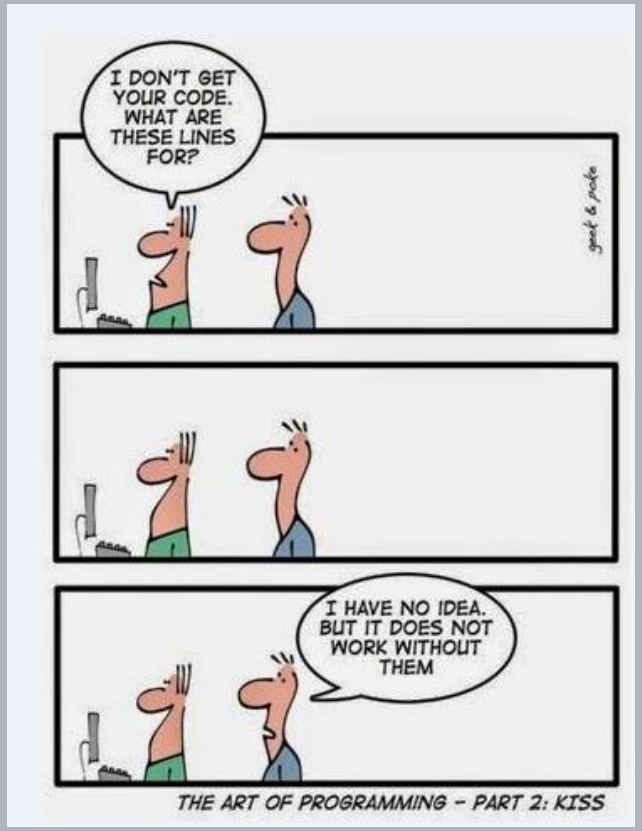
Advanced Software Design Techniques

Metrics

Prof. Dr.-Ing. Peter Fromm



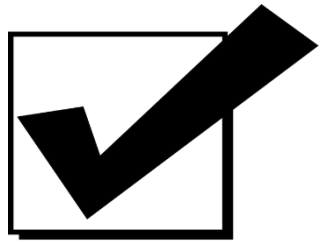
Content

- Metrics Background
- Code Metrics
  - Internal
  - External
- Design Metrics
- Process related Metrics
- Golden Rules
- Metric Tools

The maintenance loop

- Tests and (to a lesser extend) reviews require very much effort.
- Therefore they are often given a too low priority during the implementation phase!

Reduce debts at the project start ➔ architecture!



Let's write "safer C"

Static Code Analysis

Analyse code rot permanently (and automatically)

Metrics

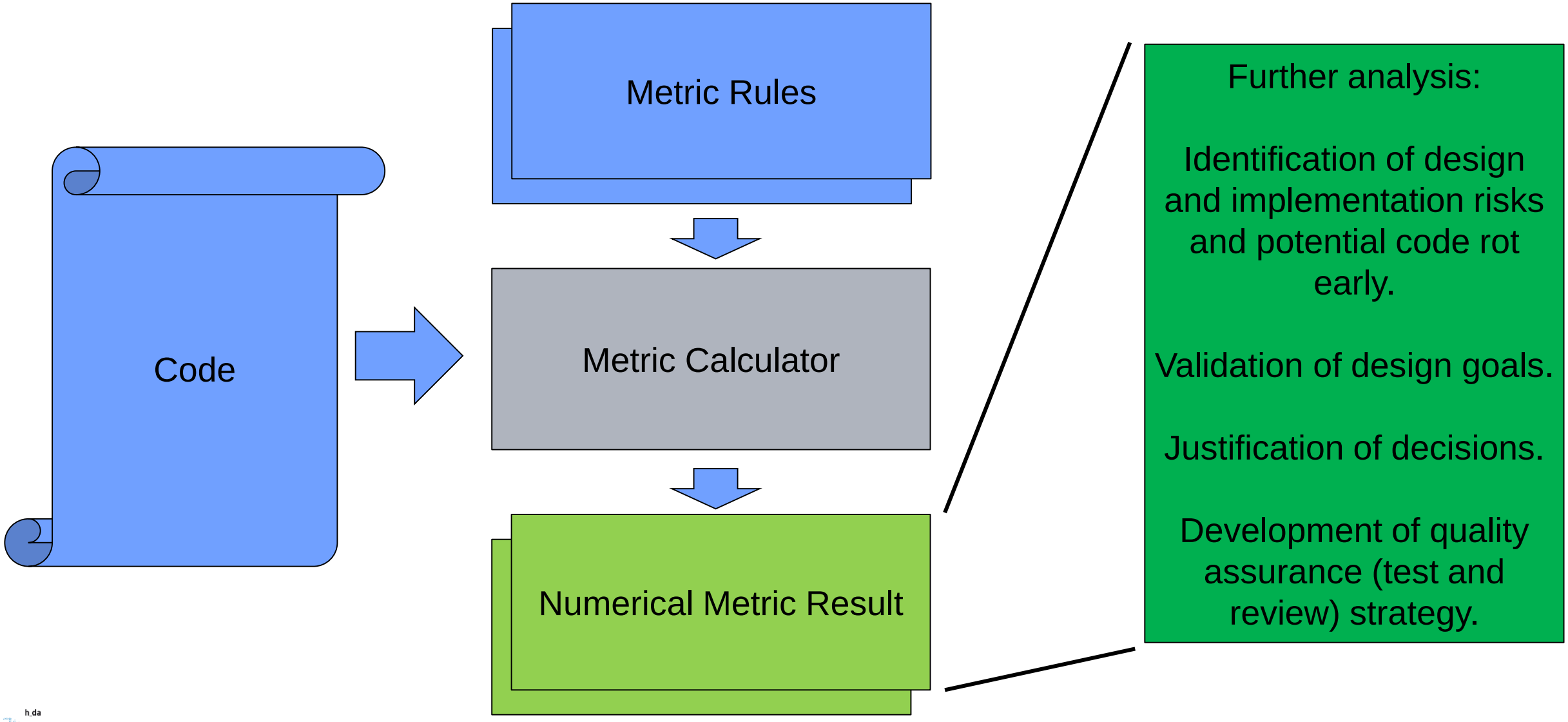
Metric Definition

A software metric is a standard of measure of a degree to which a software system or process possesses some property. Even if a metric is not a measurement (metrics are functions, while measurements are the numbers obtained by the application of metrics), often the two terms are used as synonyms. Since quantitative measurements are essential in all sciences, there is a continuous effort by computer science practitioners and theoreticians to bring similar approaches to software development. The goal is obtaining objective, reproducible and quantifiable measurements, which may have numerous valuable applications in schedule and budget planning, cost estimation, quality assurance, testing, software debugging, software performance optimization, and optimal personnel task assignments.

(Wikipedia)

Another View

Metric Calculation



In which function do we expect problems? And why?

```
unsigned int calculateTimeInMS(unsigned short hh,
   unsigned short mm, unsigned short ss)
{
   //Local variable holding the result
   unsigned int result = 0;

   //Add the individual time components
   result += hh * 3600;
   result += mm * 60;
   result += ss;

   //Translate into ms
   result *= 1000;

   //Return the final result
   return result;
}
```

```
int entryAge(int birthYear, int birthMonth, int
   entryYear, int entryMonth)
{
   int years, months;
   if ( entryMonth < birthMonth ) {
        months = 12 + entryMonth - birthMonth;
        years = entryYear - birthYear -1;
   }
   else {
        months = entryMonth - birthMonth;
        years = entryYear - birthYear;
   }
   if ( months > 5)
        years = years + 1;
   return years;
}
```

Some example metrics…

Metric	`calculateTimeInMS`	`entryAge`
CountLine	16	15
CountLineCode	9	15
CountLineComment	3	0
RatioCommentToCode	0,44	0
MaxNesting	0	1
Cyclomatic	1	3
CountInput	3	4
CountOutput	1	1

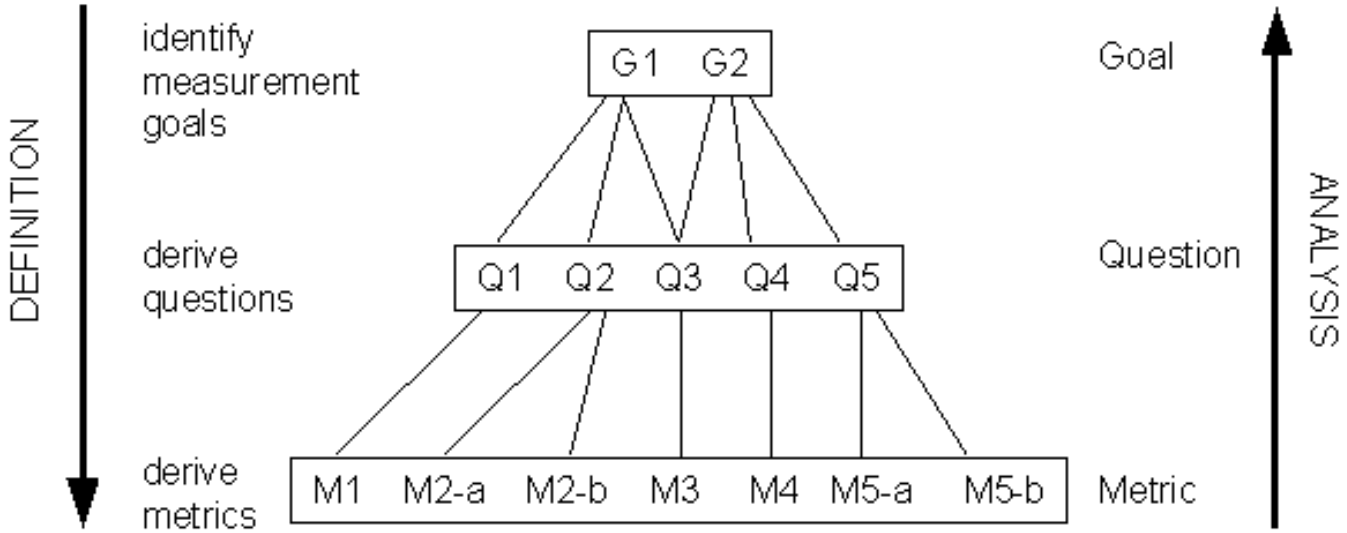
Goal / Question / Metric

The number itself does not tell us too much….

G1: The code shall be maintainable (remember IEC9126?)

Q1: Did we provide comments?
Q2: Did we choose good names?
Q3: Did we follow the KISS principle?

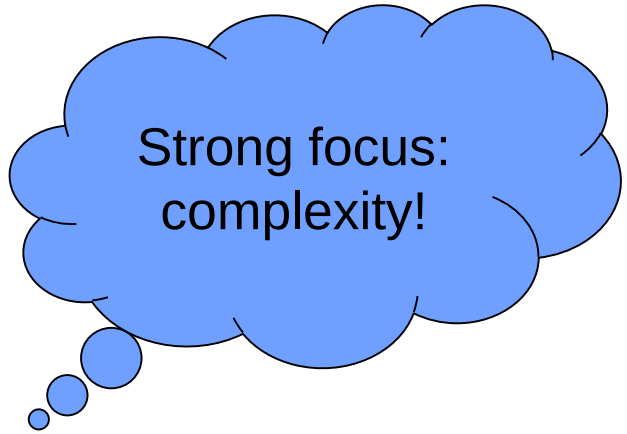
M1: Count Number of comments
M2: Calculate Comment / Code Ratio
M3: Calculate FanIn / FanOut
M4: Calculate Cyclomatic Complexity



Metrics give an indicator. And they can be compared over time. But they do not provide complete and exact answers.

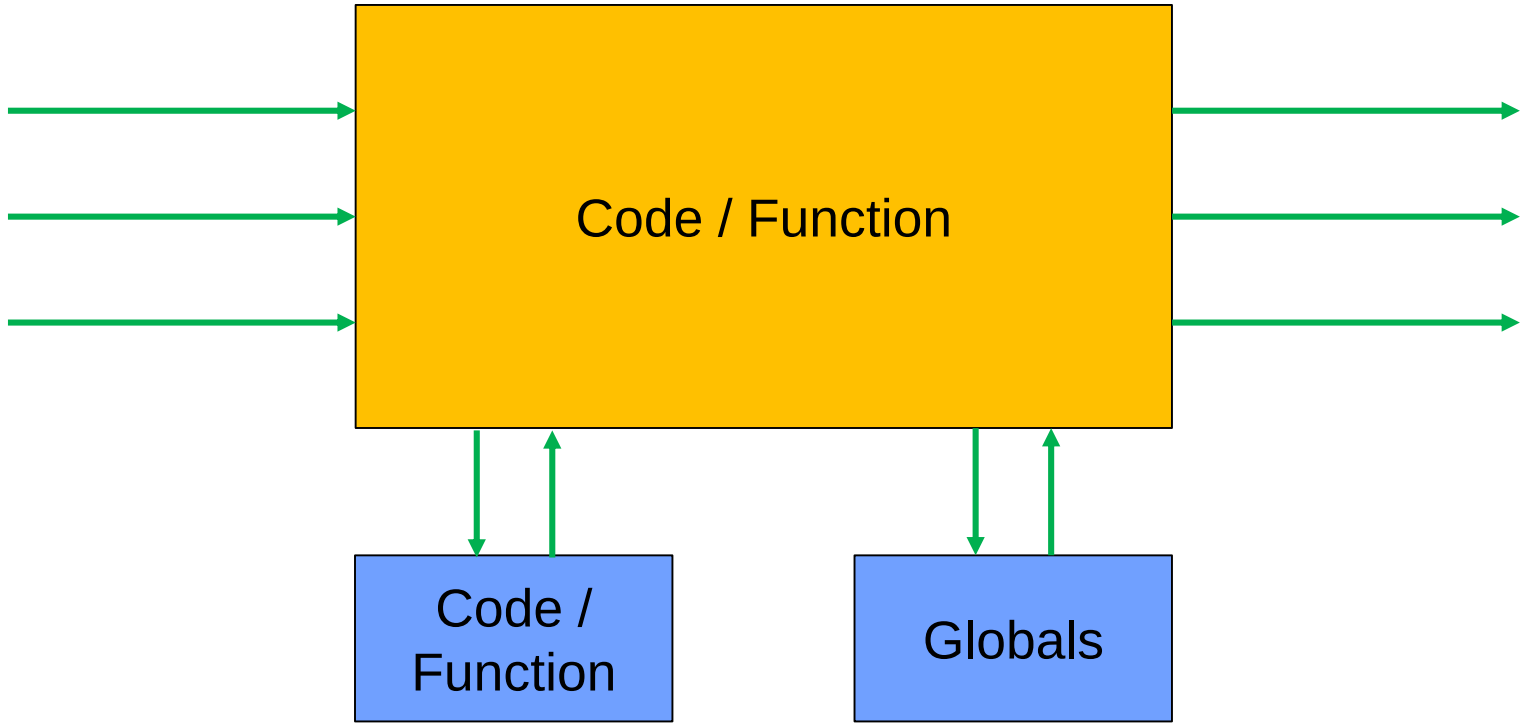
GQM and ISO9126

Goal	Question	CC - Cyclomatic Complexity	CCR - Comment / Code Ratio	FFS - Function / File Size	FI - Function FanIn	FO - Function FanOut	IAR - Inactive / Active LOC Ratio	MTB - Maintainability	NL - Nesting Level	QACWFL - QAC Warnings per file and level	RRR - RAM / ROM Report	STPC - Static Path Count	TIT - Transitive Include Tree
Functionality	Suitable						x						
	Accurate												
	Secure												
Reliability	Recoverable												
	Fault tolerant												
	Mature									x			
Usability	Understandable		x										
Efficiency	Build process												x
	Ressource										x		
	Time								x				
Maintainability	Testable	x				x		x				x	
	Analysable	x	x	x	x		x	x	x				
	Stable				x	x							x
	Changeable	x	x		x			x	x				
Portable	Co-existent												
	Adaptable												
	Replacable				x								
	Integratable					x	x						x



Internal and External Complexity Metrics

Internal_complexity = f(size, nesting, structural elements, comments, …)
External_complexity = g(data_in, data_out, function_calls,…)



Code Metric: Line Count

Goal

- Measure the size of the software to get a rough estimate about complexity, efforts for modifying, testing, redesigning etc.

Definition

- Counting the lines of code according to specific filter, build stage and scope settings
- Filter: e.g. Total LOC, Code LOC, Comment LOC, Inactive LOC, Preprocessor LOC
- Scope: Function, File, Project

Application

- Basis for estimation
- Identification of parts for refactoring / decomposition
- Often used in cooperation with other metrics

	Function	File
Warning Level	50…100	500..1000

Code Metric: Comment Code Ratio

Goal

- Is my module / function documented?
- Can somebody besides me maintain the code in a econonomic and safe manner?

Definition

- Number of Lines containing comments divided through the total number of code lines

Application

- Identify those functions which need additional documentation
- Should be used together with function size or function complexity (i.e. big/complex functions with low comment ratio typically are not maintainable)
- Complex functions or hardware functions should be documented more

	Function / File
Warning Level	10%...30%

Code Metric: Nesting Level

Goal

- Is the logic of my function too deep to be understood? Can all pathes be tested in a safe and economic manner?

Definition

- Counts the maximal depth of the function structure
  - every structural element like if, for, while,... adds one count to the metric

Application

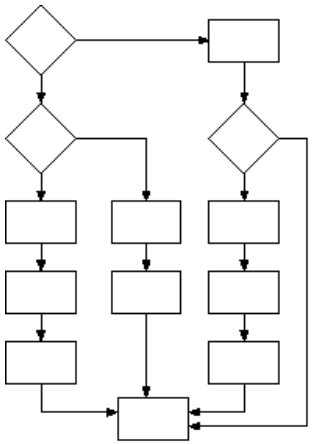
- In combination with McCabe always a winner to identify highly complex functions
- Which then should be focused on in reviews

	Function / File
Warning Level	3…5

```
// Level 0
if (a==3)
{ // Level 1
   for (b=0; b<10;b++)
   { // Level 2
   }
}
```

Code Metric: Cyclomatic Complexity or McCabe

Goal
- Is my component testable?



Definition
- Counts the number of linearly independent paths through a function
- With e edges, n nodes and p "unconnected parts of the graph" (which usually is 1) it is calculated as v(F) = e - n + 2p

e = 15
n = 13
v(F) = 15 - 13 + 2

Application
- The McCabe number implies how many branches at least must be tested (C1)
- Locate maximum complexity in file, then go for the function.
- Long switch statement are less critical – combine with Nesting Level!

	Function / File
Warning Level	5…10

Code Metric: Halstead Metric

Goal

- How complex is the development / review and how many bugs can be expected.

Definition

- n_1 = the number of distinct operators
- n_2 = the number of distinct operands
- N_1 = the number of total operators
- N_2 = the number of total operands

- Program vocabulary: $n = n_1 + n_2$
- Program length: $N = N_1 + N_2$
- Program volume: $V = N \times \log_2 n$
- Program difficulty: $D = \frac{n}{2} \times \frac{N_2}{n_2}$
- Review effort: $E = D \times V \rightarrow T = \frac{E}{18} sec$
- Expected bugs: $B = \frac{V}{3000}$

Code Metric: Halstead example

```
main()
{
    int a, b, c, avg;
    scanf("%d %d %d", &a, &b, &c);
    avg = (a + b + c) / 3;
    printf("avg = %d", avg);
}
```

$$n_1 = 12, n_2 = 7, N_1 = 27, N_2 = 15$$
$$V = 178.4$$
$$D = 12.85$$
$$E = 2292$$
$$T = 1287sec$$
$$B = 0{,}05$$

Code Metric: RAM/ROM Report

Goal

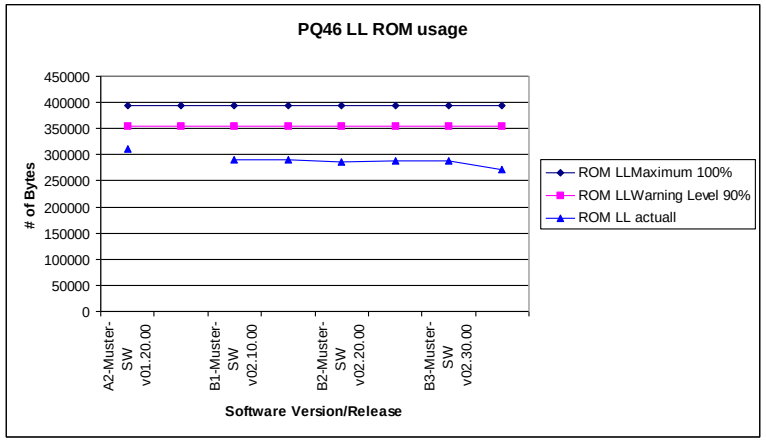
- Will the code fit into the processor resources?
  Is it possible to implement future features?

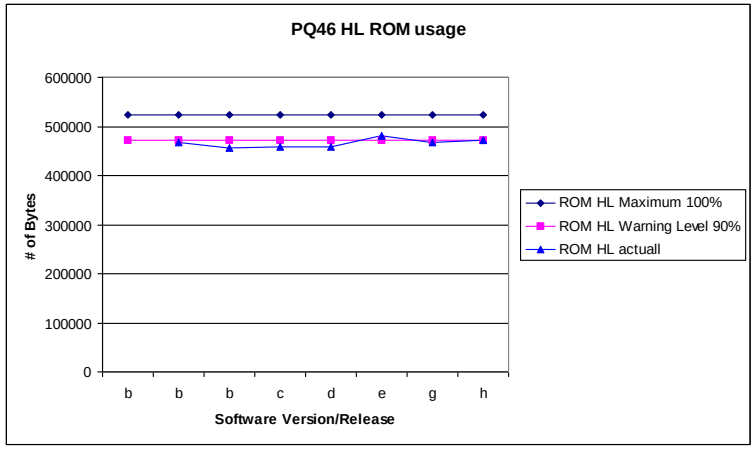
Definition

- Divide the occupied memory of a specific section
  (RAM, ROM, EEPROM, Stack) through the available resources

Application

- For estimation of the required hardware resources
- Before a reuse decision to determine, if the module
  is slim enough for the system
- Note: Stack is harder to measure!





	Project
Warning Level	70..80%

Code Metric: FanIn, FanOut

Goal

- How complex is the interface of my function / module?

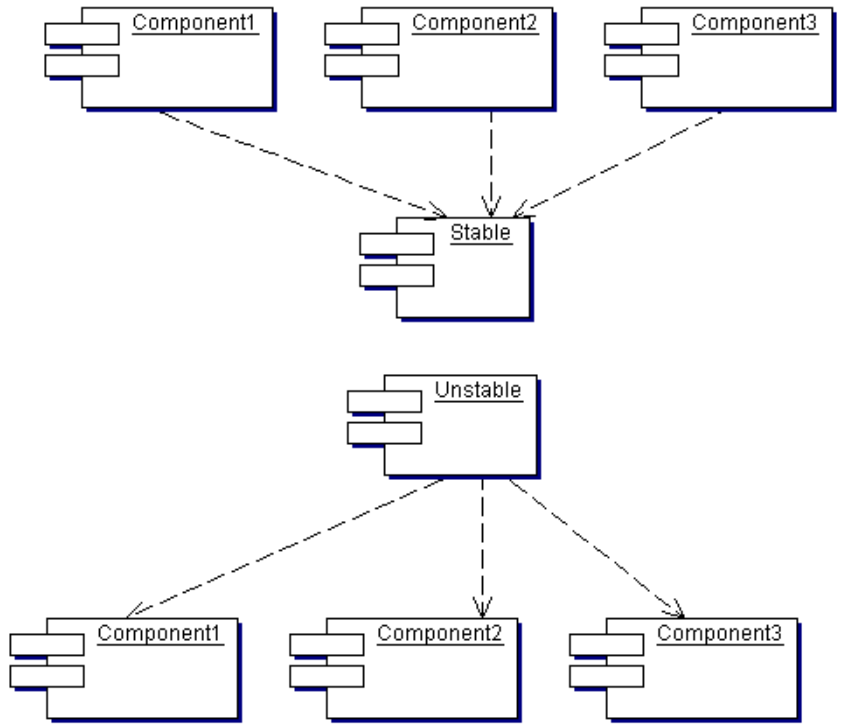
Definition

- Fan-Out: The number of outputs that are SET. This can be parameters or global variables. So Functions calls + Parameters set/modify + Global Variables set/modify.
- Fan-In: The number of inputs a function uses plus the number of unique subprograms calling the function. Inputs include parameters and global variables that are used in the function, so Functions calledby + Parameters read + Global Variables read.

Application

- High Fan-Out may be a sign for low cohesion, resulting in difficult test and integration.
- High Fan-In indicates high coupling, the module is hard to change.

Design Metric: Stability / Flexibility



The component Stable has three other components depending on it. Three good reasons not to change. On the other hand, the component Stable does no depend on any other component. It is independant and responsible.

The component Unstable depends on three other components. Very likely it will have to change. We say it is dependant and irresponsible.

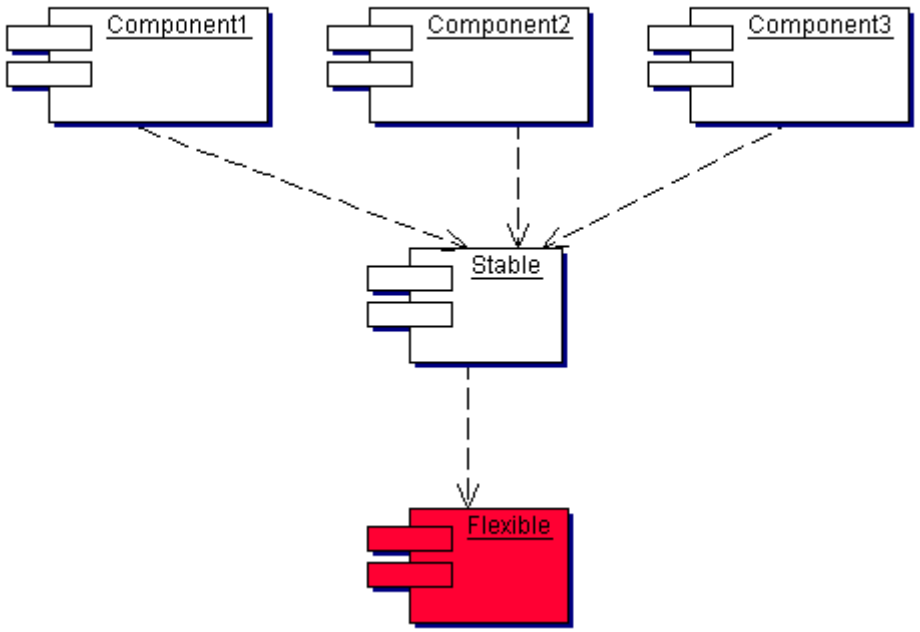
$$I = \frac{FAN_{out}}{FAN_{out}+FAN_{in}}$$

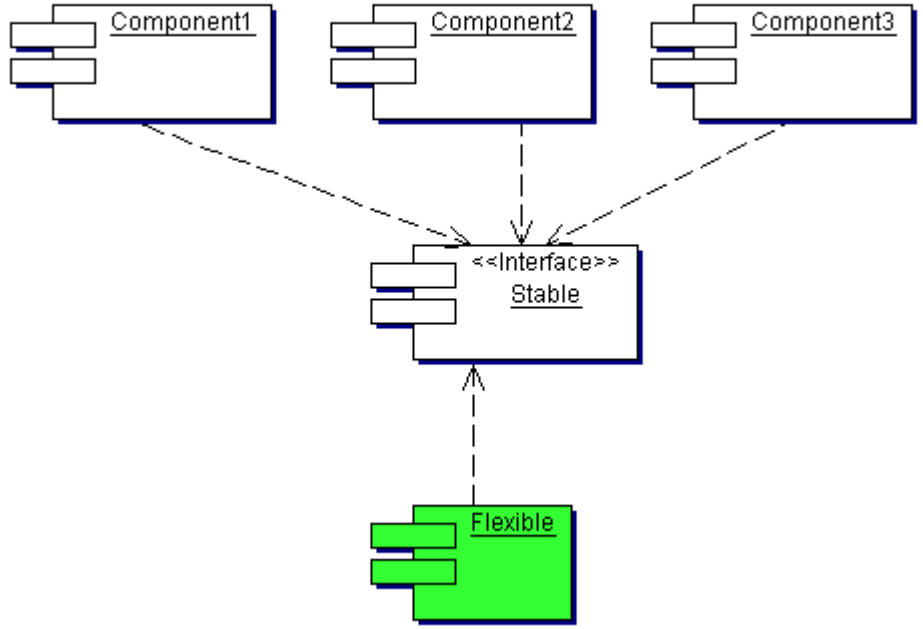
- Range: [0,1]
- 0 indicating maximum stability
- 1 indicates maximal instability

- For a simplified Fan in/out calculation we can simply count the number of #includes.

Design Metric: Stability / Flexibility

- In order to evaluate the metric values, we have to compare the calculated stability value with the given requirements.
- In case a stable component depends on one or several (flexible) components we will have a problem.
- A fixed interface or dependency inversion can solve this. In other words – the implementation can stay flexible, as long as the interface is not affected.





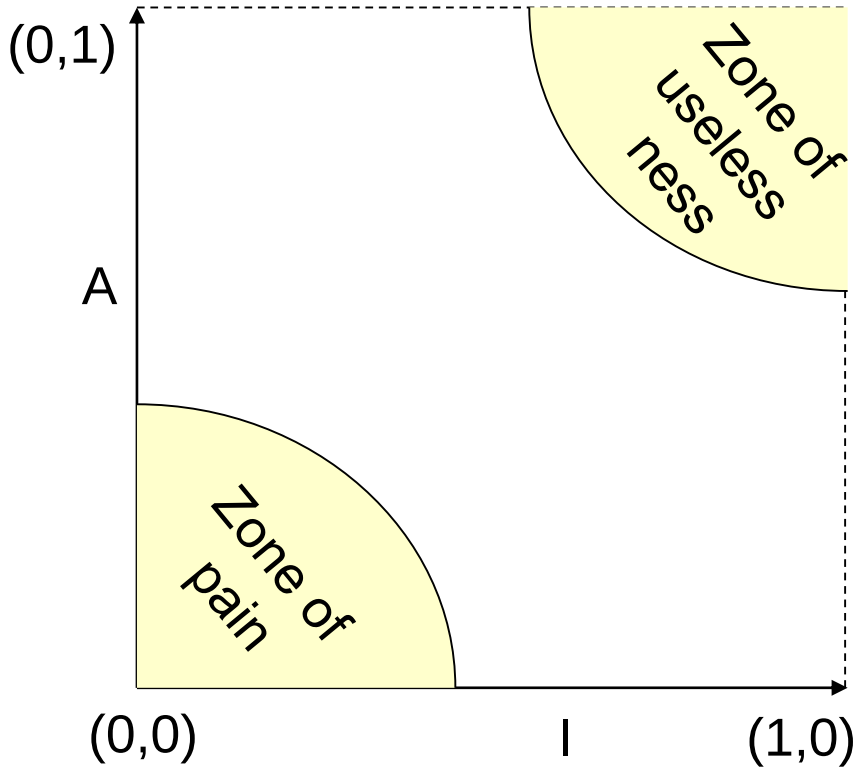
Abstraction versus instability

Instability: $I = \frac{FAN_{out}}{FAN_{out}+FAN_{in}}$

- Range: [0,1]
- 0 indicating maximum stability
- 1 indicates maximal instability

Abstraction: $A = \frac{N_a}{N_c}$

- with Na number of abstract or interface classes
- And Nc number of classes inside a component
- [0,1] : 0 no abstract classes, 1 only abstract classes



NoGo Areas

Zone of pain

- Components in this areas are highly concrete (i.e. hard to change) and we have many dependencies.
- A good example is a database schema, which is very volatile and will impact the view as well as the model components of our applications ➔ we must fix this early!
- Another example are standard libraries

Zone of uselessness

- Components in this area are highly abstract but nobody depends on them
- Very often this represents dead code, which was never implemented ➔ Refactoring

Process related Metrics

Process related metrics are focussing more on the project management side

- Count the number of testcases, reviews, change requests,…
- Track the effort / costs,…
- …

Remember the statement: Project Management is Risk Management?

Using metrics to support Project Management

- Identify complex components and make sure, that these are tested thoroughly.
- Identify components having a high FANout and make sure that they are implemented early enough and tested early enough.
- Use MCabe to get an early feeling about the minimum number of testcases you need to test a function.
- Identify stable (concrete) functions havin a high FANin and make sure they depend on stable interface. Introduce dependency inversion if needed.

Golden Rules

- Measure IN TIME to be able to define and implement corrective actions – measurement without improvement is nonsense.
- Measurement without additional analysis of the code does not provide sensible data. Instead, use the metrics to focus on parts which should be analyzed in more detail.
- Measure in regular intervals and track the – hopefully visible – improvement of the software- it's a good motivation.
- The improvement of the code must be planned. It is an additional investment for the project in the beginning – bringing benefit at the end.
- Some improvements cannot be achieved within one project but have strategic character. Involve the management for agreeing a long term strategy.

Wrap-Up

- Static code analysis and metrics are (relatively) easy and cheap.
- But the analysis requires significant effort and training!